DESIGN.md, exported 2026-07-21 (Sumatra-parity backlog: search/keynav, tabs, ebook formats, all done).

Slate — design

A suckless PDF editor, composed from proven libraries rather than a reimplemented PDF engine. Full feature set: view, annotate/markup, merge/split, redact, sign, form-fill, scan for sensitive content.

Licensing posture (real, load-bearing constraint)

`pymupdf` is AGPL-3.0/commercial dual-licensed (Artifex) — confirmed directly via its own package metadata, not assumed. Real research this session (sourced: FSF/Wikipedia's GPL material, opensource.com's AGPLv3 breakdown) converges on: copyleft/source-disclosure obligations trigger on **distribution outside your organization** (or, AGPL-specifically, letting outside users interact with a *modified* copy over a network as a service) — not on purely private/internal use. Slate is a local desktop app, not a network service, and isn't modifying PyMuPDF itself.

Devin's explicit ruling: Slate stays personal/internal-only — he will personally ensure it is never open-sourced or shared outside that. Under that constraint, per the sourced material above, the AGPL distribution trigger doesn't apply. This is not a legal ruling (get a real lawyer before treating it as one) — it's the scope decision that keeps the licensing question moot for now. If that constraint ever changes (shared with a client/vendor, open-sourced, sold), the AGPL question reopens for real and needs actual legal review at that point, plus a real look at Artifex's commercial license (no public pricing — requires contacting their sales team directly).

Why composition, not a new engine

Full ISO 32000 parity is a multi-year effort even for funded teams (PDF.js/MuPDF/Poppler-scale). The suckless answer isn't reinventing that — it's a thin app over libraries that already do each job well, matching this project's own "wrap the real tool, don't reimplement" doctrine (same pattern as Cairn's markitdown skill).

Stack

- **GUI:** Tkinter (Python stdlib) — zero extra dependency for the toolkit

  itself.

- **Rendering / annotate / merge-split / forms:** PyMuPDF (`fitz`).
- **Redaction:** PyMuPDF's `apply_redactions()` , hardened (see below).

- **Redaction verification (independent second reader):** pikepdf

  (QPDF-backed — a different codebase than PyMuPDF/MuPDF, never let the same engine that wrote the file grade its own redaction).

- **Signing:** pyHanko (real PAdES B-B/B-T/B-LT/B-LTA, Acrobat-validation

  compatible — PyMuPDF's own signature-field support is basic/visual-only, not cryptographically real).

- **Encryption / password protection:** pikepdf.

Redaction — the hardened save path (non-negotiable)

PyMuPDF's destructive-removal guarantee is conditional: physical removal only holds "assuming `Document.save()` with a suitable garbage option." `io_pdf.py`'s save path after any redaction must always:

- `garbage=4, clean=True` — never expose a faster save that skips this;

  skipping it leaves orphaned, recoverable objects in the file.

- `incremental=False` (full rewrite) — a PDF with prior incremental-update

  revisions keeps pre-redaction content byte-recoverable in its own revision history regardless of how clean the current revision is.

- Strip XMP/document metadata as part of the same hardened save — same

  underlying risk (content surviving in the wrong place) as the incremental-history issue, not a separate feature.

Real, sourced bugs exist in `apply_redactions()` (PyMuPDF GitHub #3433, #2108, #3278 — over/under-redaction in certain content-stream layouts). The test harness must catch both directions, not just "is it gone."

Signing — sequencing constraint

PDF signatures are append-only-incremental by design: signing must be the last write to a document. `io_pdf.py` must track signed-state and warn-or-refuse further edits to an already-signed document rather than silently invalidate the signature.

Forms — verified findings (slice 5, corrects the original plan)

Radio sibling-unset: the originally-cited gotcha is stale. The plan assumed PyMuPDF doesn't auto-unset sibling radio buttons and that `forms.py` would need to do it manually. Verified directly against the installed version (PyMuPDF 1.28.0): `Widget._checker()` already handles this correctly — setting one radio button's `field_value` to its own ON state and calling `.update()` automatically forces every sibling in the same field-name group back to `Off`. `forms.py` does nothing special; `tests/test_forms.py::test_radio_sibling_auto_unset` pins this behavior so a future PyMuPDF upgrade that regresses it gets caught immediately.

Real gotcha instead: radio GROUP creation, and widget page-lifetime. PyMuPDF cannot currently *create* a new interlinked radio group — adding two `Widget()` objects with a shared `field_name` via `add_widget()` raises `ValueError: bad xref` (confirmed, and matches PyMuPDF GitHub discussion #2333: group creation isn't supported, only filling an existing group's values is). Not a blocker for v1 — Slate fills received forms, it doesn't author new ones from scratch — but the test fixture's radio group had to be built at the raw PDF-object level via pikepdf instead. Separately: a `Widget` holds only a *weak* reference to its parent page; letting that page object go out of scope while still holding widgets (e.g. `forms.widgets_by_name(doc[0])` called inline, with nothing keeping `doc[0]`'s page object alive) turns a later sibling-unset into a raw `ReferenceError`, not a clean update. Always keep the page bound to a named variable for as long as any of its widgets are being read or written.

Build order (each slice has its own standalone pass/fail check)

Redaction is proven right after the minimum viewer needed to see anything — early, not deferred to "once the UI feels done."

#	Slice	Check
0	Skeleton: venv, fossil repo, Tkinter window opens a fixture PDF	Window opens, no exceptions, page count matches
1	Minimal viewer: render, nav, zoom	Every page of a fixture renders; checksums match expected
2	**Redaction core + verification harness (the trust gate)**	Plant a canary string + image in a fixture; redact; assert gone via PyMuPDF's `get_text()` AND a pikepdf raw-object scan (incl. orphaned objects) AND image byte-hash absent — on both a plain fixture and one with pre-existing incremental-update history — and explicitly assert the *unsafe* save path still shows the canary, proving the safe path is load-bearing
3	Merge/split/reorder/extract	Round-trip page count + per-page text-hash match
4	Annotate (highlight, freetext, ink, shapes, stamp)	Add one of each, reload, enumerate `page.annots()`, confirm type/rect/contents

5	Forms fill	One of each widget type; set, reload, confirm persisted; cross-check with an independent renderer; verify radio-sibling unsetting
6	Signing (pyHanko, PAdES B-B)	Sign with a self-signed test cert; verify via pyHanko's own independent validation call against the saved file
7	Security/encryption	Round-trip: correct password opens, wrong password fails, permission bits actually enforced
8	UI integration	One real end-to-end task per feature category, single sitting, no restarts

Scope decisions

OUT of v1: XFA (Adobe LiveCycle) forms (detect and warn, don't silently mis-save). PDF/A validation/conversion (unless a real compliance need surfaces — wrap veraPDF later, don't hand-roll). OCR for scanned PDFs (image-region redaction still works without it).

IN v1: opening and producing password-protected PDFs. A mandatory safe-save policy — never overwrite the original in place; auto-backup or save-as-copy before any destructive op, especially redaction.

Business question, not engineering: PAdES B-LT/B-LTA needs a real non-self-signed cert to matter to external recipients — v1 ships B-B (fine for internal sign-off), B-LT/LTA only worth building once a real signing cert exists.

Scan for sensitive content (`scan.py`)

Started as a one-off scratch audit script (Devin asked to scan his real Downloads folder for anything needing redaction), promoted to a real feature after it found genuine sensitive content in a real file (a bank account + routing number in an actual UMB account-verification letter) and, separately, had a real false-negative during its own development.

Scope: number-shaped financial/PII patterns only — SSN (`123-45-6789`), ABA routing numbers, labeled account numbers, and Luhn-valid credit card numbers. Does NOT catch general PII (a resume's phone/address with no account-shaped context), business-confidential content, or anything on an image-only/scanned page — same OCR gap already named out-of-v1 for redaction itself. A page with 0 extractable characters is reported as its own `unscannable` finding rather than silently folded into "nothing found" — those are different claims, and conflating them is exactly the bug this project caught in itself (next paragraph).

Real bug, caught auditing Devin's actual Downloads folder, not a hypothetical: the first version matched a label and its value on the *same line* (`Account Number:\s*(\d+)`). Real PDFs routinely don't lay text out that way — the bank letter's own extracted text put `Account Number:`, a blank line, and `9825039777` as three separate lines. The same-line version silently reported the real file clean. Fixed by

scanning forward a few non-blank lines after a label match instead of requiring the value on the same line; pinned as its own named regression test (`test_label_and_value_on_separate_lines_regression`) building that exact three-line layout, not folded into a general-case test.

UI wiring: Edit → "Scan this document..." runs `scan.scan_document` against the open file and offers to mark every hit with a resolvable rect as a pending redaction in one click — scan and redact are meant to chain together, not be two disconnected features. File → "Scan folder for sensitive PDFs..." reuses `scan.scan_directory` for the batch-audit case (the actual real-world use this started from).

Real bug found wiring this into the UI, unrelated to scan.py itself: `_on_release` 's redact-mode branch called a blocking `messagebox.showinfo` on *every single drag* — meant as "region marked" feedback, but it made a multi-region redaction pass annoying (a modal popup per mark) and, worse, made this exact code path hang in an automated test once nothing was there to dismiss the dialog. Fixed by removing the popup entirely — the status bar (`render()`) already shows the pending-redaction count for the current page, which is the correct non-blocking feedback.

Text editing — real v2 feature, gated by design (in progress)

Editing existing body-paragraph text was in Devin's original ask, missed in the first pass of this plan (view/annotate/merge-split/redact/sign/ form-fill/scan only). Real, deliberate scope call from Devin once this gap was pointed out: ship it as a **separate, gated capability**, not a plain menu item everyone gets — "so not just everyone has PDF text editing, but everything else they would need without editing a 'final form' of a document." The rest of v1 stays open to everyone; text editing sits behind a passphrase gate, off by default.

Why this is genuinely a separate feature: editing existing PDF body text means re-flowing text runs and matching the original font/kerning exactly — PDF is page-fixed glyph positions, not a reflow format. Even Adobe/Foxit's own "Edit Text" is imperfect at this. Real approach: redact-then-reinsert (`add_redact_annot` + `apply_redactions()` + `insert_text` at the same spot) — **with the redaction fill set to white/transparent, not black**. Real bug caught live running the slice-0 experiment below: reusing `redact.py` 's `mark_region()` as-is produced a solid black bar instead of new text, because that function is correctly built for actual redaction (black fill on purpose) — wrong default for this feature. Text-editing's own redact call uses `fill=(1, 1, 1)` .

Three-tier font-safety approach (refined from the original two-tier plan, Devin's idea): before falling to a crude Base-14 substitute, check whether the same font is already installed as a real system font first — most business PDFs use Calibri/Arial/Times New Roman/Segoe UI, already sitting on the same Office-standardized Windows machines that would run Slate. Real, non-approximated glyphs for the common case, zero bundled fonts, zero new dependencies.

1. **reusable** — font fully embedded, not subsetted (`page.get_fonts()` ,

   no `ABCDEF+` prefix) → reuse via `extract_font` / `insert_font` .

2. **system-font** — `fontmatch.find_system_font()` (new module,

   Windows via stdlib `winreg` checking HKCU then HKLM with name normalization, verified via `fitz.Font(fontfile=...).name`; Linux via `fc-match` for dev, with a **real, live-confirmed pitfall**: `fc-match` never fails, it always substitutes a "closest" font — `fc-match "Calibri"` on this dev box, which has no Calibri, returned `DejaVu Sans` silently. Must compare the *returned* family against what was asked for, reject a mismatch as "not really installed.")

3. **substitute-needed** — Base-14 mapped from the font's flags bitfield

   (serif/bold/italic/monospace), same as the original plan. Only this tier needs an up-front warning in the UI — 1 and 2 are both real fonts.

Slice 0 (the font-fidelity experiment) — done, real images, not a prediction anymore. Built a fixture with a genuinely embedded, non-subsetted font (confirmed via `get_fonts()`: `ext='ttf'`, `basefont='DejaVu Serif Book'`, no subset prefix), then rendered all three tiers after a redact+reinsert cycle:

- **Reuse:** visually identical to the original — same serif shapes,

  same weight, same spacing, as expected for literally the same font program.

- **System-font:** clean, correct glyph rendering — a real typeface,

  not a wrong-shaped approximation (rendered a different real font in this demo to make the point: a genuine font file always renders correctly, regardless of which one it is).

- **Substitute:** renders, but visibly different letterforms (serif

  style, spacing, descenders) from the original — confirms the predicted degradation is real, not alarmist.

Gate mechanism: `hashlib.pbkdf2_hmac("sha256", ..., 600_000)`, local salted hash (not plaintext) at `~/.slate/unlock.json` (same convention as `recent.py`), stdlib-only. First-run (Devin confirmed): clicking "Edit Text" with none set yet prompts to set one right there, no separate admin step. Unlock is session-only. Stated plainly: a local UX gate, not real access control.

Slice 1 (`fontmatch.py`) — done, 6/6 tests, real fc-match substitution pitfall confirmed and guarded against.

Slice 2 (`textedit.py` core) — done, 9/9 tests. Two real bugs caught writing these tests, not before:

- `font_safety`'s original match compared `page.get_fonts()`'s `name`

  field against `span["font"]` — but `name` is the page's font- *resource* alias (e.g. `"F1"`, whatever key insert_font/the PDF producer picked), while `span["font"]` (from `get_text("dict")`) is the font's own internal name. Confirmed live: these are different strings even for a font this exact code just

embedded itself (`"F1"` vs `"DejaVuSerif"`). This would have made tier 1 (reuse) silently never fire on *any* real document — every edit would have fallen to tier 2/3 even when the exact original font was genuinely reusable. Fixed: compare `span["font"]` against `basefont` (the font's real reported name, subset-prefix stripped), normalized with `fontmatch`'s existing `_normalize_font_name` (already built, already tested) — reused rather than duplicated, and it already handles the registry-vs-PostScript-style naming mismatch this needed too (`"DejaVu Serif Book"` vs `"DejaVuSerif"`).

- `edit_text` called `page.insert_font()` for the reusable/system-font

  tiers *before* `apply_redactions()`. `apply_redactions()` rebuilds page resources and drops any font not yet referenced by the content stream — so the just-registered font vanished before `insert_text` could use it (`Exception: need font file or buffer`). Fixed by moving font registration to *after* the redaction call, immediately before the actual `insert_text`.

Slice 3 (`gate.py`) — done, 6/6 tests. `hashlib.pbkdf2_hmac`, 600k iterations, random salt, stored at `~/.slate/unlock.json`. Covers correct unlock, wrong passphrase, re-set invalidating the old one, fail-closed when nothing's set yet, and plaintext never touching disk.

Slice 4 (UI wiring in `slate.py`) — done, 5/5 tests. Feature complete, all 4 slices built and tested (79/79 total suite). "Edit Text (locked, click)..." on the Edit menu: first click ever prompts to set a passphrase right there (no separate admin step); set-but-locked prompts to unlock; unlocked routes straight into textedit mode. Unlock is session-only, re-locks on restart -- a local UX gate, not real access control, stated plainly in the code. A canvas click in textedit mode runs `detect_span` + `font_safety`, surfaces the tier-3 substitute-font warning in the edit dialog when relevant, and catches `TextFitError` as a real message instead of a crash.

Fixtures

`tests/fixtures/` holds small synthetic PDFs only — generated programmatically, never real documents. One fixture must include pre-existing incremental-update history (to catch the redaction/history gap above) and one must include a pre-existing tag tree (to confirm redact/merge/annotate don't silently strip it, even though authoring tagged PDF is out of v1). Same discipline applies to the one synthetic epub fixture used below — built at test time via `zipfile`, never committed as a binary.

Sumatra-parity backlog — done (v3)

Devin's real question: after using SumatraPDF's TOC/recent-files panel as this project's own UX reference already, what else is worth peeling from it? Three concrete, real slices, in cheapest-to-most-structural order:

1. **`search.py` + keyboard nav** — in-document Find (`/` or View >

Find...), `Return` / `Shift-Return` / `n` / `N` step through matches with wraparound, current match highlighted red vs. yellow for the rest (canvas overlay, redrawn every `render()`, not real annotations). `j` / `k` page nav, `g` / `G` jump to first/last page. Reuses PyMuPDF's own `page.search_for()` (already case-insensitive, confirmed live). Real gotcha: `search_for("")` returns `None`, not `[]` — guarded explicitly. All single-letter bindings are guarded on "is an Entry currently focused" so they don't hijack literal search text (`n`, `g` etc. are all valid characters to search for).

2. **Tabs** — `tab.py`'s `Tab` is a plain per-document state container

   (path/doc/viewer/page/mode/pending_redactions/search_state). `ttk.Notebook` is used purely as a tab-*selector strip* — each "tab" is a never-shown placeholder child frame; the single existing shared toolbar/canvas/find-bar/toc keeps doing all the actual rendering, unchanged. On switch, the outgoing tab's mutable fields save back into its `Tab`, the incoming tab's fields load into the same flat `self.doc` / `self.path` /etc. attributes every pre-existing method already reads — so none of those ~30 methods needed touching, and all 94 pre-tabs tests kept passing unchanged (a single open document is just the one-tab case). Real bug: selecting a `Notebook` tab only fires `<<NotebookTabChanged>>` on the next idle-loop pass, not synchronously — fixed via a `_select_tab()` helper that calls the handler directly (idempotent alongside the real event, for actual interactive clicks).

3. **Ebook formats** (EPUB/MOBI/FB2/CBZ/TXT/MD) — confirmed via

   PyMuPDF's own docs feature matrix *and* hands-on (a real synthetic epub through the unmodified `viewer.py`): PyMuPDF/MuPDF already opens all of these natively. Zero new dependency, almost no new code — mostly widening `open_file()`'s dialog filter. CHM is *not* supported (not in the matrix; would need PyMuPDF Pro) — left out. Devin's call: PDF-only Edit/File menu items (redact, sign, forms, encrypt, merge/split, save) are disabled whenever the active tab's document isn't a real PDF (`doc.is_pdf`), via `_update_pdf_only_menu_state()` hooked into the tab-switch path — automatically correct across tab switches, no extra wiring. Real bug caught writing this slice's epub test: `_title()` unconditionally ran `sign.is_signed()` (pyHanko) against `self.path`, which crashed with "Illegal PDF header" the instant a non-PDF path was opened — fixed by gating that check on `doc.is_pdf` too (signing is itself a PDF-only action; a non-PDF is never "signed" by definition).

Explicitly still out of scope: true ebook-reader extras (TTS, adjustable reflow/font size for epub, night mode) — these were a separate, earlier exploratory research thread (real Piper TTS sizing numbers gathered, nothing built) and stay that way unless asked for directly.